

CoppeliaSim Simulation Report

Title of Assessment : 7-DOF Manipulator Motion Planning

Task : 1

Name : Sarath Kumar Komathukattil

University ID : K2454461

Module code : ME7028

Table of Contents

1. Scenario description	3
2. Assigned Task.....	3
3. Motion Planning and Execution Strategy	3
4. Control Script Architecture	4
5. Visual Execution.....	5
6. Summary and Reflection	6
7. Appendix A- Lua script with section wise explanation	6

1. Scenario description

The simulated environment consists of a 7-degree-of-freedom redundant robotic manipulator operating within CoppeliaSim. The environment has four obstacles, one of which is a floor plane, all configured as a part of the collision geometry. The manipulator's structure follows a recognisable scene tree hierarchy. Despite the default naming, this arrangement facilitates the script to constantly access joint handles by index, which simplifies configuration setup and ensures motion commands are executed well. This setup mimics the practical use in confined industrial environment.

2. Assigned Task

The main objective is to autonomously move the robot's end-effector through a sequence of four predefined spatial target points. These targets are implemented using dummy objects in the simulation. A successful solution must avoid contact with all environmental obstacles, which include the floor. By making use of a hybrid planning approach, the system must detect feasible inverse kinematics (IK) configurations and plan collision-free paths between the configurations.

3. Motion Planning and Execution Strategy

To accomplish the task, a hybrid strategy was implemented that integrates IK solving with OMPL-based motion planning. For each target, an IK configuration is calculated using the simIK module by sampling feasible joint positions that achieve the target position. The sampled configurations are then tested for validity, including obstacle and floor collision.

OMPL is used to plan a motion trajectory once a valid IK solution is determined. The RRTConnect algorithm assesses a collision-free path between the current joint state and the IK-derived goal configuration. This path is then executed step-by-step using a simulation stepping mode.

Collision avoidance is ensured using:

- A validation callback that checks sampled configurations for collisions with the environment and evaluates the z-axis height of the end-effector.
- Grouped robot and obstacle collections, where obstacles include three static objects and the floor.

This dual-layer validation ensures that the robot safely traverses through the environment without any collision.

4. Control Script Architecture

The Lua control script integrates CoppeliaSim's simulation interface with OMPL and simIK libraries to create a modular planning pipeline. Below are the major components used:

- **getConfig() / setConfig():**
getConfig() gets the robot's current joint positions, while setConfig() updates joint states to visualise the computed path.
- **configurationValidationCallback():**
This function discards invalid configurations by checking for robot-obstacle collisions and ensures a minimum tip height.
- **IK Environment Setup:**
The robot's joint hierarchy is mapped into an IK environment using simIK.createEnvironment() and simIK.addElementFromScene(), enabling conversion between Cartesian targets and joint configurations.
- **Target Pose Processing:**
For each target, the position relative to the base is obtained and then implemented in the IK environment. This approach facilitates repeatable planning cycles for multiple targets.
- **IK Configuration Search:**
simIK.findConfigs() performs up to 1000 trials to determine a feasible joint configuration that obtains the desired position. Configurations that pass the collision check are only used.
- **Trajectory Playback:**
The simulation is set to stepped mode using sim.setStepping(true) to enable controlled animation of the robot's motion. Each joint configuration along the planned path is applied in sequence, advancing the simulation one step at a time.

5. Visual Execution

Figures below display the key moments in the simulation:

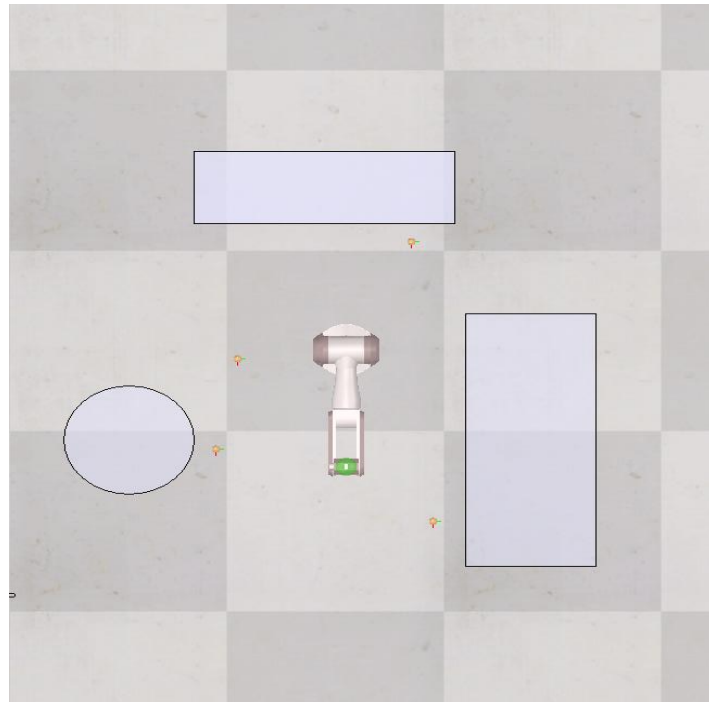


Figure 1: Initial configuration of the manipulator at the start of the simulation.

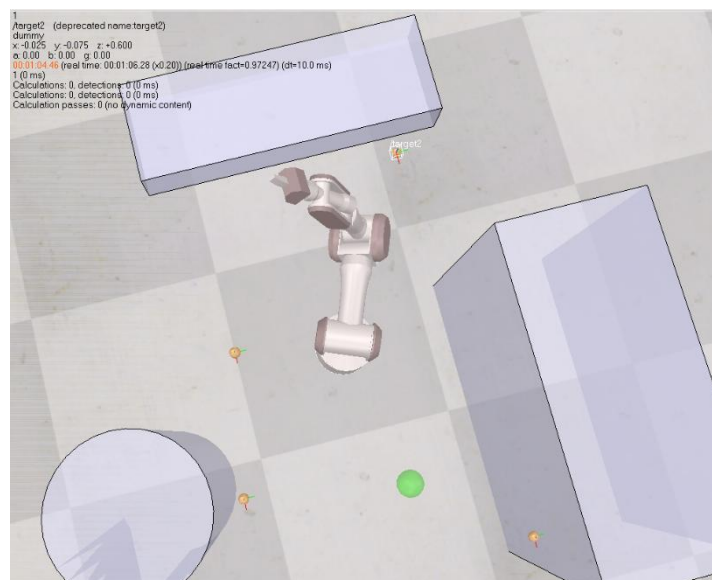


Figure 2: The manipulator navigates to the second target by evading obstacles

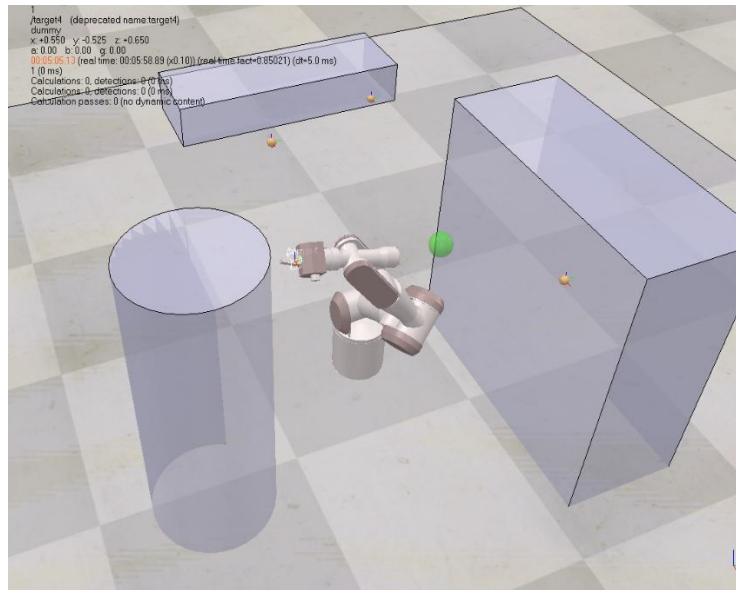


Figure 3: The manipulator reaching the final target position.

6. Summary and Reflection

The task was successfully carried out using a simulation-driven strategy that integrates both inverse kinematics and sampling-based motion planning. The robot manoeuvred to all targets without contacting the environment through effective collision validation and modular scripting.

The coursework forms a strong foundation for manipulator planning in complex environments. Future improvements could enable effective obstacle handling for more complex collaborative scenarios.

7. Appendix A- Lua script with section wise explanation

- **Import Required Libraries:** These modules provide access to simulation control, inverse kinematics, and OMPL-based path planning.

```
sim = require 'sim'
simOMPL = require 'simOMPL'
simIK = require 'simIK'
```

- **Helper Functions: getConfig() and setConfig():** These functions retrieve and apply joint configurations.

```
function getConfig()
    local retVal = {}
    for i = 1, #simJointHandles, 1 do
        retVal[i] = sim.getJointPosition(simJointHandles[i])
    end
    return retVal
end

function setConfig(config)
    for i = 1, #simJointHandles, 1 do
        sim.setJointPosition(simJointHandles[i], config[i])
    end
end
```

- **Collision and Validity Check: configurationValidationCallback():** This function checks whether a proposed configuration causes any collision or results in the robot's end-effector being too low (below a 1 cm safety margin above the ground).

```
function configurationValidationCallback(config)
    local tmp = getConfig()
    setConfig(config)

    local valid = true

    if sim.checkCollision(robotCollection, obstacleCollection) ~= 0 then
        valid = false
    end

    local tipZ = sim.getObjectPosition(sim.getObject('../tip'), -1)[3]
    if tipZ < 0.01 then
        valid = false
    end

    setConfig(tmp)
    return valid
end
```

- **Main Control Logic: sysCall_thread():** This threaded function drives the entire planning and execution pipeline. It sets up the robot structure, target list, IK and OMPL environments, and loops through each target for path planning and movement.

```
function sysCall_thread()
```

- **Joint and Target Initialization:** Initializes joint handles and identifies target dummy objects. Also sets up projection flags for OMPL.

```

simBase = sim.getObject('..')
simJointHandles = {}
useForProjection = {}
for i = 1, 7, 1 do
    simJointHandles[i] = sim.getObject('../joint', {index = i - 1})
    useForProjection[i] = i <= 3 and 1 or 0
end

local simTip = sim.getObject('../tip')
local simTarget = sim.getObject('../target')
targets = {
    sim.getObject('/target1'),
    sim.getObject('/target2'),
    sim.getObject('/target3'),
    sim.getObject('/target4')
}

```

- **Robot and Obstacle Collection Creation:** Creates object collections used in collision checking during motion planning.

```

robotCollection = sim.createCollection()
sim.addItemToCollection(robotCollection, sim.handle_tree, simBase, 0)

obstacleCollection = sim.createCollection()
sim.addItemToCollection(obstacleCollection, sim.handle_single, sim.getObject('/obstacle1'), 0)
sim.addItemToCollection(obstacleCollection, sim.handle_single, sim.getObject('/obstacle2'), 0)
sim.addItemToCollection(obstacleCollection, sim.handle_single, sim.getObject('/obstacle3'), 0)
sim.addItemToCollection(obstacleCollection, sim.handle_single, sim.getObject('/obstacle4'), 0)

```

- **IK Environment Setup:** Creates an IK environment and maps the robot joints and targets to the IK solver.

```

ikEnv = simIK.createEnvironment()
ikGroup = simIK.createGroup(ikEnv)
local _, simToIkObjectMapping = simIK.addElementFromScene(
    ikEnv, ikGroup, simBase, simTip, simTarget, simIK.constraint_position
)

ikJointHandles = {}
for i = 1, #simJointHandles, 1 do
    ikJointHandles[i] = simToIkObjectMapping[simJointHandles[i]]
end
ikTarget = simToIkObjectMapping[simTarget]
ikBase = simToIkObjectMapping[simBase]

```

- **Main Target Navigation Loop:** Loops through each of the four targets, computes a valid IK solution, plans a path using OMPL, and executes the motion step-by-step in the simulation.


```

local tcnt = 1
while true do
    local pose = sim.getObjectPose(targets[tcnt], simBase)|
    simIK.setObjectPose(ikEnv, ikTarget, pose, ikBase)

    local configs = simIK.findConfigs(ikEnv, ikGroup, ikJointHandles, {
        maxDist = 1.5,
        trials = 1000,
        cb = configurationValidationCallback
    })

    if #configs > 0 then
        local config = configs[1]

        local task = simOMPL.createTask('task')
        simOMPL.setAlgorithm(task, simOMPL.Algorithm.RRTConnect)
        simOMPL.setStateSpaceForJoints(task, simJointHandles, useForProjection)
        simOMPL.setCollisionPairs(task, {robotCollection, obstacleCollection})
        simOMPL.setStartState(task, getConfig())
        simOMPL.setGoalState(task, config)
        simOMPL.setup(task)

        local res, path = simOMPL.compute(task, 6, -1, 1000)

        if res and path then
            local sw = sim.setStepping(true)
            for i = 1, simOMPL.getPathStateCount(task, path) do
                local conf = simOMPL.getPathState(task, path, i)
                setConfig(conf)
                sim.step()
            end
            sim.setStepping(sw)
            sim.wait(1)
        end

        simOMPL.destroyTask(task)
    end

    tcnt = tcnt + 1
    if tcnt > 4 then tcnt = 1 end
    sim.wait(1)
end
end

```